AI Masters
Алгоритмы-2024 зима
семестровая контрольная (final)
Никита Загайнов

1[3] Рассмотрим следующую реализацию вычисления суммы n элементов. Сначала первый элемент складывается со вторым, третий с четвертым и так далее попарно. Потом этот процесс повторяется с появившимися парами и так далее, пока не останется одно число. Все числа в начале k-битовые, сложение линейно по битовой длине. По мере выполнения алгоритма длина чисел растет. Оцените сложность выполнения этого алгоритма.

Решение

Время алгоритма: $T(n,k) = T(\frac{n}{2}, k+1) + nk$ Докажем, что итоговая асимптотика $O(n(k+\log n))$. Очевидно, что максимальное значение параметра k равно $k + \log_2 n$ на последнем вызове функции. Принимая во внимание этот факт, имеем:

$$T(n,k) = O(n(k+\log n) + \frac{n}{2}(k + \log\frac{n}{2}) + \ldots + (k+\log n)) = O(n(k+\log n))$$

2[1 + 1]

1. $T(n) = T(n-3) + n$
2. $T(n) = 27T(\frac{n}{3}) + n^4$

Решение

1. $$T(n) = n + (n-3) + (n-6) + \ldots + 3 = \sum_{i=1}^{n/3} 3i = O(n^2)$$

2. Применим третий случай мастер-теоремы:

$$n^4 = \Omega(n^{\log_3 27+\varepsilon}) = \Omega(n^{3+\varepsilon})$$

И

$$27\left(\frac{n}{3}\right)^4 = 27 \cdot \frac{n^4}{81} = \frac{1}{3}n^4 < cn^4, \quad c > \frac{1}{3}$$

Тогда по третьему случаю мастер-теоремы:

$$T(n) = \Theta(n^4)$$

3[1] Оцените асимптотику числа печатей.

```
def f(n):
    if (n < 100):
        return

    f(n / 2); f(n / 2); f(n / 2)

    for i = 0; i < n; i += 1:
        for m = 1; m < i**2; m *= 2:
            print("hehe")
```

Решение

Количество печатей:

$$T(n) = \begin{cases} 0, & n < 100 \\ 3T(n/2) + n\log n, & n \geq 100 \end{cases}$$

Очевидно, что $T(n) = O(\bar{T}(n))$ для

$$\bar{T}(n) = 3\bar{T}(n/2) + n\log n$$

По первому случаю мастер-теоремы:

$$n\log n = O(n^{\log_2 3-\varepsilon}), \quad \varepsilon < \log_2 3 - 1$$

Тогда асимптотика будет:

$$T(n) = \bar{T}(n) = O(n^{\log_2 3})$$

4[2] Алхимики работают с n веществами, превращая их друг в друга. Вам дают алхимическую книгу, в которой описаны эти превращения. Предложите алгоритм, позволяющий найти все такие вещества, которые можно получить из вещества A и такие, что из каждого из них можно получить A.

Решение

Превращения в книге представимы в виде графа, где вершины – вещества, а ребра – превращения. Тогда чтобы найти все необходимые вещества, нужно запустить `DFS` из вершины A в исходном графе и в обратном графе. В результате комбинируем те вершины, которые были посещены, тогда получим искомое множество. Этот алгоритм работает, так как первый обход найдёт все вещества, которые могут получиться из A (по построению `DFS`), а второй обход найдёт все вещества, из которых можно получить A (по построению обратного графа).

Сложность – $O(|V| + |E|)$, $|V|$ - количество веществ, $|E|$ - количество превращений. (`DFS` - $O(|V| + |E|)$, Обращение графа - $O(|E|)$).

5[3] Вы с коллегой решили найти кратчайшие пути из вершины s во все остальные в ориентированном ациклическом графе с положительными целыми весами, ограниченными сверху константой W, в котором степени всех вершин не менее 3. Коллега разделил все ребра на единичные и массу промежуточных вершин, не нашел пути, ушел обедать, а потом домой. Предложите алгоритм, который вернет граф в первоначальное состояние, докажите его корректность и оцените асимптотику.

Решение

Так как в исходном графе степени всех вершин не менее 3, то нам нужно убрать все вершины степени 2, они были добавлены искусственно. Это гарантированно вернёт граф в исходное состояние, так как при введении промежуточных вершин, мы добавляем только вершины степени 2. Сделать это можно следующей модификацией `DFS`: хранить последнюю рассмотренную вершину, у которой была степень 3, и соединять её с настоящей вершиной, если её степень тоже 3. Далее удалить все вершины степени 2. Мы не создадим новых рёбер, так как если две вершины степени 3 соединены ребром, алгоритм добавит уже существующее ребро (Его за новое не считаем), в обратном случае, ребро будет добавлено между парой вершин степени 3, увеличивая их степень и не влияя на степени промежуточных вершин.

Сложность – $O(W|V| + W|E|)$ – сложность обхода в графе, где после введения искусственных вершин добаывилось не болле $W|V|$ вершин и $W|E|$ рёбер.

6[4] Предложите алгоритм, принимающий на вход матрицу смежности неориентированного графа с единичными весами и вычисляющий матрицу весов кратчайших путей для всех пар вершин.

Решение

Воспользуемся алгоритмом Флойда-Варшалла, который высчитывает длины кратчайших путей путём обновления исходной матрицы смежности d_{ij}:

$$d_{ij} = \min(d_{ij}, d_{ik} + d_{kj})$$

для всех i, j, k. Алгоритм корректен, так как он обновляет расстояние между каждой парой вершин i, j через промежуточную вершину k для каждой вершины k.

Сложность – $O(|V|^3)$, где $|V|$ – количество вершин в графе.

7[4] На вход задачи поступает набор $\{(x_i, y_i, z_i, r_i)\}_{i=1}^n$. Каждая четверка чисел задает шар с центром в x_i, y_i, z_i радиуса r_i. Постройте алгоритм, проверяющий, можно ли пройти из точки x_1, y_1, z_1 в точку x_2, y_2, z_2, в каждый момент времени оставаясь внутри шаров. Докажите его корректность и оцените асимптотику.

Решение

Для решения задачи построим граф, где вершины – центры шаров, рёбра соединяют только те шары, которые пересекаются (расстояние между центрами не больше суммы радиусов). Далее запустим `DFS` из точки x_1, y_1, z_1 и проверим, достигаема ли точка x_2, y_2, z_2. Алгоритм корректен по построению `DFS`. Чтобы построить граф, потребуется проверить расстояние между каждой парой шаров, что займёт $O(|V|^2)$ времени. `DFS` работает за $O(|V| + |E|)$.

Сложность алгоритма – $O(|V|^2 + |E|)$.

8[4] Известно, что в ориентированном графе с целыми весами есть ровно один несамопересекающийся отрицательный цикл, который при этом является стоком. Предложите алгоритм, который получает на вход описание графа и выводит число, которое нужно прибавить к весам всех ребер, чтобы убрать отрицательный цикл. Докажите его корректность и оцените асимптотику.

Решение

Так как цикл один и он является стоком, его можно найти обходом графа (если алгоритм повторно посетил какую-то из вершин). Далее, обходим найденный цикл, считая его вес, и возвращаем $\max(-w, 0)$. Алгоритм корректен, так как цикл несамопересекающийся и у него только один вариант обхода.

Сложность алгоритма – $O(|V| + |E|)$ (два обхода графа).

9[5] Рыбак взял с собой лодку, но забыл весла. Выйти из положения он намеревается с помощью прочного спиннинга, на катушке которого намотано l метров прочной лески. Дело происходит в Карелии, так что на озере много (n) островов. Рыбак может бросить блесну на соседний остров, если он не дальше l метров, и подтянуться туда путем сматывания удочки. Ему нужно добраться до конкретного острова, поскольку там лучше всего клюет. На одном из островов лежит вторая катушка длины $L > l$. Постройте алгоритм, позволяющий найти кратчайший путь до целевого острова, докажите его корректность и оцените асимптотику. Карта озера известна, остров с катушкой известен. Начать передвижение можно в любой точке берега, но после этого ступать на берег нельзя, поскольку там ходит рыбнадзор и проверяет лицензии.

Решение

Для начала построим два графа и одно множество. В обоих графах вершинами будут острова, которые соединены рёбрами, если расстояние между ними не превышает l или L (для первого и второго графа). В множество положим все все острова, до которых можно добраться от берега. Далее запустим `DFS` из целевой вершины в первом графе. Если досягаема хотя бы одна вершина из множества, то рыбак может добраться без новой лески. Если нет, то запускаем `DFS` во втором графе, проверяя досягаемость острова с новой леской, а потом запускаем его в первом графе из этого острова с леской, проверяя, можно ли добраться до одного из островов из множества. Если оба раза получилось, то целевой остров досягаем, если нет, то нет. Алгоритм корректен по построению `DFS`.

Сложность алгоритма – $O(|V| + |E|)$, так как мы запускаем обход не более 3 раз.